

Práctica 1: Preparamos el entorno de trabajo

1. Introducción: herramientas, paquetes y plataformas

Para empezar a trabajar necesitaremos distintas herramientas que servirán de gran ayuda en el desarrollo de la aplicación.

Además, se tendrán que utilizar otras herramientas en el servidor una vez que la aplicación se ejecute.

Imprescindibles para comenzar:

- **Un navegador:** Firefox, Chrome, Opera, etc.
- **Un entorno de desarrollo integrado (IDE):** Nos proporciona herramientas de reconocimiento de clases avanzado, integración con el SGBD, etc. Ejemplos:
 - PHPStorm
 - Visual Studio
 - NetBeans
 - Eclipse PDT
 - *Si tienes pocos recursos en tu máquina, y no puedes instalar un IDE, puedes usar un editor de código como, por ejemplo, Sublime Text. O, mejor aún, puedes usar:*
 - El editor online de W3Schools
 - Un entorno de pruebas como PHP SandBox
- **Conocer un lenguaje de programación web** como PHP
- **Un servidor de bases de datos:**
 - MySQL
 - MariaDB
- **Un servidor web:**
 - **Remoto:** podemos usar un hosting gratuito
 - **Local:** será lo más cómodo para empezar a trabajar al principio, ya que sólo tendremos que descargar e instalar el paquete apropiado para nuestro equipo.

Cómo montar nuestro propio servidor local

Montar nuestro propio servidor local con Apache, un gestor de bases de datos (MariaDB/MySQL), y PHP puede ser tedioso por lo que podemos recurrir a paquetes de software y plataformas que nos faciliten el trabajo

a) Utilizando paquetes software:

- **Linux:** Si trabajamos en Linux, la solución será una arquitectura **LAMP** (Linux Apache MySQL PHP).



- **Windows:** Si vas a trabajar en Windows puedes usar **WampServer**.



- **Multiplataforma:** Si no lo tienes claro y prefieres un paquete que pueda instalarse tanto en Linux como en Windows utiliza **XAMPP**.



En cualquiera de ellos se incluye un servidor web Apache y un servidor de bases de datos MySQL o MariaDB.

b) Otras alternativas:

- **Docker:** Crear un entorno local independiente (o contenedor) con las especificaciones exactas para el proyecto usando Docker.



- **GitHub Codespaces:** Usar un entorno de desarrollo basado en la nube como GitHub Codespaces.



2. Instalar el IDE

Deberás **solicitar la licencia de estudiante** en la página del JetBrains:

<https://www.jetbrains.com/shop/eform/v2/students> usando tu cuenta @g.educaand.es o con una copia de tu matrícula.

Regístrate y entra en la página oficial con tus credenciales.

Descarga PhpStorm y sigue los pasos del asistente en pantalla.

Abre PhpStorm una vez finalizada la instalación e inicia sesión con las credenciales de tu cuenta de JetBrains aprobada para activar tu licencia de estudiante automáticamente

3. Preparandonos para instalar Docker Desktop en Windows 11

Este curso utilizaremos **Docker**, una herramienta fundamental que nos permite aislar el entorno y garantizar que el código se ejecute de la misma forma para todo el alumnado.

Durante las prácticas, tu ordenador funcionará únicamente como interfaz para editar el código y visualizar los resultados en el navegador. Sin embargo, la ejecución propiamente dicha —el servidor web, PHP y la base de datos MySQL— correrá dentro de contenedores gestionados mediante un archivo `docker-compose.yml`.

Así evitaremos conflictos locales en tu equipo y prevenimos el clásico problema de *"en mi máquina no funciona"*. Todos tenéis la misma versión de PHP y las mismas extensiones.

3.1. Dónde escribes vs. dónde se ejecuta

Esta es una de las partes que suele generar más dudas al principio, por lo que es fundamental tenerla clara desde el primer momento:

1. **Tu código vive en tu equipo:** Las carpetas y archivos de tu proyecto están almacenados localmente en tu ordenador y los editas con tu herramienta habitual (PHPStorm, VS Code ...) exactamente igual que siempre.
2. **Sincronización en tiempo real:** Esa carpeta de tu ordenador está conectada al contenedor Docker, de modo que cualquier cambio que guardes en un archivo como `index.php` en tu equipo es visto por el contenedor al instante, sin necesidad de copiar ni exportar nada manualmente.
3. **Visualización en el navegador:** Para ver el resultado de tu aplicación, no debes hacer doble clic en el archivo para abrirlo directamente en el navegador (si lo haces, solo verás el texto plano del código). Debes acceder a través de la dirección web proporcionada por el servidor del contenedor (por ejemplo, `http://localhost:8080`, según la configuración de tu `docker-compose.yml`).

En resumen: escribes fuera, se ejecuta dentro y lo visualizas en el navegador.

Nota importante: Utilizarás Docker a nivel operativo para tus proyectos, sin profundizar en la administración avanzada de sistemas, temática que corresponde al módulo de Despliegue.

Recuerda: si en tu equipo no puedes usar Docker siempre puedes recurrir a XAMPP. Tendrás las instrucciones para su instalación en la plataforma Moodle.

3.2. Qué pondremos en marcha (Windows 11)

Construiremos un entorno **100% en Docker** con:

Componente	
PHP	8.5, con Apache integrado (<code>php:8.5-apache</code>)
MySQL	8.4
phpMyAdmin	última imagen oficial
Composer	última versión (se actualiza sola en cada build)
Xdebug	3.x, configurado para PhpStorm y VS Code a la vez
Apache	<code>mod_rewrite</code> , <code>mod_ssl</code> , <code>mod_headers</code> y <code>mod_vhost_alias</code> activos; <code>.htaccess</code> habilitado (<code>AllowOverride All</code>)
PDO MySQL / MySQLi	ambas extensiones activas
Git, zip/unzip	instalados dentro del contenedor
HTTPS	certificado local wildcard de confianza con mkcert (<code>*.localhost</code>), sin avisos del navegador
PHPUnit	instalado vía Composer, listo en cada proyecto
Multi-proyecto	subdominio propio y automático por proyecto (<code>https://<proyecto>.localhost/</code>), sin modificar Apache al crear uno nuevo

Docker es el único sitio donde se ejecuta realmente PHP; PhpStorm y VS Code solo editan código y depuran contra ese PHP

Subdominio dinámico con `mod_vhost_alias`

En vez de crear un fichero de configuración de Apache por cada proyecto (y tener que recargar Apache cada vez), usamos **un único vhost comodín** con la directiva `VirtualDocumentRoot`. Apache traduce automáticamente cualquier `proyectoX.localhost` a la carpeta `proyectoX/public` correspondiente, sin ninguna acción manual:

```
https://proyecto1.localhost/ → /var/www/projects/proyecto1/public/  
https://proyecto2.localhost/ → /var/www/projects/proyecto2/public/  
https://nuevo.localhost/    → /var/www/projects/nuevo/public/
```

Los navegadores modernos (Chrome, Edge, Firefox) resuelven **cualquier** subdominio `*.localhost` a `127.0.0.1` de forma nativa (RFC 6761), así que tampoco hace falta tocar el fichero `hosts` de Windows.

Por qué el dominio `localhost` (sin subdominio) va a una carpeta aparte

Es importante que `https://localhost/` (sin subdominio de proyecto) **no** apunte a la misma carpeta que los proyectos. Si lo hiciera, cualquiera podría entrar directamente a `https://localhost/proyecto1/composer.json`, `.../src/AlgunaClase.php` o `.../tests/` y ver o ejecutar archivos que se supone que quedan fuera del acceso web (esa es la razón de tener `public/` separada de `src/` en cada proyecto). Por eso:

- `https://localhost/` → una página de aviso, en una carpeta vacía e independiente (`/var/www/default`).
- `https://<proyecto>.localhost/` → **solo** la carpeta `public/` de ese proyecto (`/var/www/projects/<proyecto>/public`).

Estructura de carpetas en Windows:

Cómo organizaremos los proyectos

```

C:\proyectos\docker-php\
├── docker\
│   ├── Dockerfile
│   └── apache\
├── docker\certs\
├── docker-compose.yml
├── projects\
│   ├── proyecto1\
│   │   ├── public\
│   │   │   ├── index.php
│   │   │   └── .htaccess
│   │   ├── src\
│   │   ├── tests\
│   │   └── composer.json
│   └── proyecto2\
│       └── public\
└── new-project.ps1
comando
    
```

← certificado wildcard generado por mkcert

← ÚNICO punto servido por la web

← fuera del alcance del navegador

← crea un proyecto completo en un solo comando

3.3 Requisitos del sistema

- Windows 11 de 64 bits, cualquier edición (Home sirve).
- Virtualización activada en la BIOS/UEFI (en la mayoría de portátiles ya viene activada).

3.4 Activar WSL2

1. Abre el **menú Inicio**, escribe **PowerShell**, clic derecho → **Ejecutar como administrador**.
2. Ejecuta:

```
wsl --install
```

3. Reinicia el ordenador cuando lo pida.
4. Si se abre una ventana para crear un usuario Linux (Ubuntu), crea uno cualquiera; no se usa directamente en este entorno.
5. Comprueba, en una PowerShell normal (no administrador):

```
wsl --status
```

Debe indicar **Versión predeterminada: 2**.

4. Descargar e instalar Docker Desktop

1. Ve a <https://www.docker.com/products/docker-desktop/> y descarga la versión para Windows.

2. Ejecuta el instalador dejando marcada **"Use WSL 2 instead of Hyper-V"**.
3. Reinicia si te lo pide.
4. Abre **Docker Desktop** desde el menú Inicio y acepta los términos de servicio.
5. Espera a que el icono de la ballena de la barra de tareas quede estable ("Docker Desktop is running").

4.1 Verificar que funciona

```
docker --version
docker compose version
docker run hello-world
```

⚠ Si aparece un error tipo `failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine`, el daemon no ha arrancado todavía: ejecuta `wsl -shutdown`, espera 10 segundos y vuelve a abrir Docker Desktop antes de continuar.

5. Crear la estructura de carpetas del proyecto

```
mkdir C:\proyectos\docker-php
cd C:\proyectos\docker-php
mkdir docker
mkdir docker\apache
mkdir docker\certs
mkdir projects
```

Ábrela con el explorador de archivos o con VS Code (code `C:\proyectos\docker-php`) para ir creando los ficheros siguientes.

6. El Dockerfile

Crea `C:\proyectos\docker-php\docker\Dockerfile`:

```
FROM php:8.5-apache

# --- Paquetes del sistema: git, zip/unzip, librerías para extensiones ---
# --- Paquetes del sistema: git, zip/unzip, librerías para extensiones ---
RUN apt-get update && apt-get install -y \
    git \
    zip \
    unzip \
    libzip-dev \
    libpng-dev \
    libjpeg62-turbo-dev \
    libfreetype6-dev \
    libonig-dev \
    libxml2-dev \
    zlib1g-dev \
    default-mysql-client \
    curl \
    && rm -rf /var/lib/apt/lists/*

# --- Extensiones PHP: GD de forma independiente ---
RUN docker-php-ext-configure gd --with-freetype --with-jpeg \
    && docker-php-ext-install gd

# --- Resto de extensiones PHP ---
# --- Extensiones PHP ---
RUN docker-php-ext-configure gd --with-freetype --with-jpeg && docker-php-ext-
install gd
RUN docker-php-ext-install pdo_mysql
RUN docker-php-ext-install mysqli
RUN docker-php-ext-install zip
RUN docker-php-ext-install mbstring
RUN docker-php-ext-install exif

# --- Xdebug (para PhpStorm y VS Code) ---
RUN pecl install xdebug

# --- Módulos de Apache: reescritura, SSL, headers y subdominios dinámicos ---
RUN a2enmod rewrite ssl headers vhost_alias
# --- Composer: última versión, copiada de la imagen oficial ---
COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

# --- Configuración de Apache (paso 4) ---
COPY apache/000-default.conf /etc/apache2/sites-available/000-default.conf
COPY apache/default-ssl.conf /etc/apache2/sites-available/default-ssl.conf
RUN a2ensite default-ssl

# --- Página de aviso para https://localhost/ (sin subdominio de proyecto) ---
COPY apache/default-index.html /var/www/default/index.html

# --- Configuración de PHP para desarrollo ---
COPY apache/php.ini-development-overrides.ini /usr/local/etc/php/conf.d/zz-dev-
overrides.ini
```

```
# --- Xdebug: configuración común para cualquier editor ---
COPY apache/xdebug.ini /usr/local/etc/php/conf.d/xdebug.ini

# --- PHPUnit disponible globalmente dentro del contenedor ---
RUN composer global require phpunit/phpunit --no-interaction \
    && ln -s /root/.composer/vendor/bin/phpunit /usr/local/bin/phpunit

WORKDIR /var/www/projects

EXPOSE 80 443
```

7. Configuración de Apache

7.1 Vhost HTTP (docker\apache\000-default.conf)

```
# Vhost para http://localhost (sin subdominio): solo una página de aviso
<VirtualHost *:80>
    ServerName localhost
    DocumentRoot /var/www/default

    <Directory /var/www/default>
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>

# Vhost dinámico: http://<proyecto>.localhost ->
/var/www/projects/<proyecto>/public
<VirtualHost *:80>
    ServerName proyectos.localhost
    ServerAlias *.localhost
    VirtualDocumentRoot /var/www/projects/%1/public

    <Directory /var/www/projects>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

7.2 Vhost HTTPS (docker\apache\default-ssl.conf)


```

<IfModule mod_ssl.c>

<VirtualHost *:443>
    ServerName localhost
    DocumentRoot /var/www/default

    <Directory /var/www/default>
        Require all granted
    </Directory>

    SSLEngine on
    SSLCertificateFile      /etc/ssl/certs/wildcard-localhost.pem
    SSLCertificateKeyFile    /etc/ssl/private/wildcard-localhost-key.pem
</VirtualHost>

<VirtualHost *:443>
    ServerName proyectos.localhost
    ServerAlias *.localhost
    VirtualDocumentRoot /var/www/projects/%1/public

    <Directory /var/www/projects>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    SSLEngine on
    SSLCertificateFile      /etc/ssl/certs/wildcard-localhost.pem
    SSLCertificateKeyFile    /etc/ssl/private/wildcard-localhost-key.pem
</VirtualHost>

</IfModule>

```

Apache prioriza siempre el `ServerName` exacto (`localhost`) sobre el `ServerAlias` comodín (`*.localhost`) cuando ambos vhosts comparten IP:puerto, así que no hay conflicto entre los dos bloques.

7.3 Página de aviso (`docker\apache\default-index.html`)

```

<!DOCTYPE html>
<html lang="es">
<body style="font-family: sans-serif; padding: 2rem;">
    <h1>Entorno Docker PHP 8.5</h1>
    <p>Este dominio no sirve proyectos directamente.</p>
    <p>Accede a cada proyecto por su propio subdominio, por ejemplo:
        <code>https://proyecto1.localhost/</code></p>
</body>
</html>

```

7.4 Ajustes de PHP para desarrollo (docker\apache\php.ini-development-overrides.ini)

```
; Reporte de errores para desarrollo
display_errors = On
display_startup_errors = On
log_errors = On
error_reporting = E_ALL

; Recursos
memory_limit = 512M
max_execution_time = 120
max_input_time = 120

; Subida de archivos y formularios
upload_max_filesize = 64M
post_max_size = 64M
max_input_vars = 5000

; Zona horaria y codificación
date.timezone = Europe/Madrid
default_charset = "UTF-8"

; Sesiones (2 horas, para no perder el trabajo durante una clase larga)
session.gc_maxlifetime = 7200
```

7.5 Configuración base de Xdebug (docker\apache\xdebug.ini)

```
zend_extension=xdebug
xdebug.mode=debug,develop
xdebug.start_with_request=yes
xdebug.client_host=host.docker.internal
xdebug.client_port=9003
xdebug.idekey=DOCKERR
xdebug.discover_client_host=false
```

Esta configuración es única para todo el contenedor y por tanto vale para **todos** los proyectos a la vez: no hay que tocar nada al crear un proyecto nuevo.

8. Certificado HTTPS wildcard de confianza con mkcert

8.1 Instalar mkcert

```
winget install FiloSottile.mkcert
```

Cierra y vuelve a abrir PowerShell para que reconozca el comando.

8.2 Instalar la autoridad certificadora local

```
mkcert -install
```

Acepta el aviso de permisos de administrador si aparece: añade la CA de mkcert al almacén de confianza de Windows.

8.3 Generar el certificado wildcard

```
cd C:\proyectos\docker-php\docker\certs
mkcert -key-file wildcard-localhost-key.pem -cert-file wildcard-localhost.pem
"*.localhost" localhost 127.0.0.1 ::1
```

Esto genera `wildcard-localhost.pem` y `wildcard-localhost-key.pem`, válidos para `localhost`, cualquier `*.localhost` (presente o futuro), `127.0.0.1` e `::1`.

9. El archivo docker-compose.yml

Crea `C:\proyectos\docker-php\docker-compose.yml`:

```
services:
  web:
    build:
      context: ./docker
    container_name: php85_web
    ports:
      - "80:80"
      - "443:443"
      - "9003:9003"
    volumes:
      - ./projects:/var/www/projects
      - ./docker/certs/wildcard-localhost.pem:/etc/ssl/certs/wildcard-localhost.pem:ro
      - ./docker/certs/wildcard-localhost-key.pem:/etc/ssl/private/wildcard-localhost-key.pem:ro
    depends_on:
      - db
    networks:
      - php-net
    extra_hosts:
      - "host.docker.internal:host-gateway"

  db:
    image: mysql:8.4
    container_name: php85_db
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: app
      MYSQL_USER: dev
      MYSQL_PASSWORD: dev
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - php-net

  phpmyadmin:
    image: phpmyadmin:latest
    container_name: php85_phpmyadmin
    restart: unless-stopped
    environment:
      PMA_HOST: db
      PMA_USER: root
      PMA_PASSWORD: root
    ports:
      - "8080:80"
    depends_on:
      - db
    networks:
      - php-net
```

```
volumes:
  db_data:

networks:
  php-net:
```

Nota clave: `./projects` se monta en `/var/www/projects`, **no** en `/var/www/html` ni en `/var/www/default`. La carpeta `/var/www/default` (con la página de aviso) queda dentro de la propia imagen, copiada en el `Dockerfile` (paso 3) — así ningún proyecto puede "aparecer" accidentalmente al entrar por `localhost` sin subdominio.

9.1 Levantar el entorno

```
cd C:\proyectos\docker-php
docker compose up -d --build
```

9.2 Comprobación

- `https://localhost/` → página de aviso, sin error de certificado.
- `http://localhost:8080` → phpMyAdmin, usuario `root`, contraseña `root`.

10. Configurar Xdebug en PhpStorm y en VS Code

Mapeo de rutas único y válido para todos los proyectos:

- **Ruta local (Windows):** `C:\proyectos\docker-php\projects`
- **Ruta remota (Docker):** `/var/www/projects`

10.1 PhpStorm

Importante: el entorno Docker **siempre** se arranca por terminal con `docker compose up -d`.

Ninguna configuración dentro de PhpStorm arranca Apache/MySQL por ti. PhpStorm solo se conecta al PHP que ya está corriendo en el contenedor para ofrecer autocompletado, ejecutar PHPUnit y depurar. Si alguna vez ves `ERR_CONNECTION_REFUSED` al recargar un proyecto, lo primero es comprobar `docker compose ps` en la terminal, no revisar la configuración de PhpStorm.

1. Crear el servidor con el path mapping:

- Menú **Run** → **Edit Configurations...** → botón **+** → **PHP Web Page**. Dale un nombre, por ejemplo `docker`.
- En el campo **Server**, pulsa el botón `...` situado justo al lado del desplegable: se abre el diálogo de servidores.
- Pulsa **+** y crea uno con:
 - Name: `docker`
 - Host: `localhost`
 - Port: `443`

- Marca **Use path mappings** y enlaza `C:\proyectos\docker-php\projects` con `/var/www/projects`.
 - Acepta, vuelve a "PHP Web Page", selecciona el servidor **docker** recién creado y pon como **Start URL** la del proyecto que quieras abrir (p. ej. `https://proyecto1.localhost/`).
- 2. **Settings** → **Languages & Frameworks** → **PHP** → **Debug**: comprueba que el puerto Xdebug incluye **9003** y que "Can accept external connections" está marcado (en versiones recientes ya viene así por defecto).
- 3. **Settings** → **Languages & Frameworks** → **PHP**: añade un intérprete remoto + → **From Docker, Vagrant, VM, WSL, Remote...** → **Docker Compose** → `docker-compose.yml`, servicio **web**. Sirve para **todos** los proyectos a la vez y es solo para inspección de código/PHPUnit, no arranca el contenedor.
- 4. Activa "Start Listening for PHP Debug Connections" (icono en la barra de herramientas).
- 5. Pon un breakpoint en `public/index.php` de cualquier proyecto y, con el contenedor ya arrancado por terminal, pulsa ► sobre la configuración "PHP Web Page" que creaste en el punto 1 (o simplemente recarga la URL a mano en el navegador si ya tenías la escucha activada).

Si alguna vez da "Port 9003 is busy" al lanzar esa configuración: significa que ya había una escucha de Xdebug activa de una sesión anterior que no se cerró bien. Pulsa el icono para apagar la escucha y vuelve a activarla; no hace falta reiniciar Docker por esto.

10.2 VS Code

1. Instala **PHP Debug** (Xdebug.php-debug) y **PHP Intelephense**.
2. Crea `C:\proyectos\docker-php\.vscode\launch.json`:

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Escuchar Xdebug (Docker)",
      "type": "php",
      "request": "launch",
      "port": 9003,
      "pathMappings": {
        "/var/www/projects": "${workspaceFolder}/projects"
      }
    }
  ]
}
```

3. Abre la carpeta raíz `C:\proyectos\docker-php`, ve a **Run and Debug**, elige "Escuchar Xdebug (Docker)" y pulsa ▶. Sirve para cualquier proyecto sin tocar nada más, gracias al mapeo genérico `/var/www/projects ↔ projects/`.

11. Composer y PHPUnit dentro de cada proyecto

Todo se ejecuta **dentro del contenedor**, no en Windows:

```
docker compose exec -w /var/www/projects/proyecto1 web composer install
docker compose exec -w /var/www/projects/proyecto1 web phpunit tests
```

O entrando directamente a una terminal del contenedor:

```
docker compose exec web bash
cd /var/www/projects/proyecto1
composer require monolog/monolog
phpunit --version
```

12. Crear un proyecto nuevo en un solo comando

Crea `C:\proyectos\docker-php\new-project.ps1`:

```

param(
    [Parameter(Mandatory=$true)]
    [string]$Nombre
)

# Escribe texto en UTF-8 SIN BOM (Out-File -Encoding utf8 añade BOM en
# Windows PowerShell 5.1, y esa marca invisible rompe el parser JSON de
# Composer y puede causar errores raros en PHP). Se usa esta función para
# cualquier fichero de texto/código del proyecto.
function Write-Utf8NoBom {
    param([string]$Path, [string]$Content)
    $utf8NoBom = New-Object System.Text.UTF8Encoding $false
    [System.IO.File]::WriteAllText($Path, $Content, $utf8NoBom)
}

$ruta = "C:\proyectos\docker-php\projects\$Nombre"

if (Test-Path $ruta) {
    Write-Host "El proyecto '$Nombre' ya existe en $ruta" -ForegroundColor Red
    exit 1
}

# --- Estructura de carpetas (separando lo público de lo privado) ---
New-Item -ItemType Directory -Path "$ruta\public" | Out-Null
# New-Item -ItemType Directory -Path "$ruta\src" | Out-Null
# New-Item -ItemType Directory -Path "$ruta\tests" | Out-Null

# --- index.php de entrada en public/ ---
$indexPhp = @"
<?php
require_once __DIR__ . '/../vendor/autoload.php';
echo '<h1>Proyecto: $Nombre</h1>';
echo '<p>PHP ' . phpversion() . '</p>';
"@
Write-Utf8NoBom "$ruta\public\index.php" $indexPhp

# --- .htaccess en public/ con mod_rewrite ya activo (front controller) ---
@"
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
"@ | Out-File -Encoding ascii "$ruta\public\.htaccess"

# --- composer.json en la raíz del proyecto ---
<#
$composerJson = @"
{
    "name": "clase/$Nombre",
    "type": "project",
    "require": {},
    "require-dev": {

```



```

        "phpunit/phpunit": "^11.0"
    },
    "autoload": {
        "psr-4": { "App\\": "src/" }
    }
}
"@
Write-Utf8NoBom "$ruta\composer.json" $composerJson
#>

# --- Git y Composer dentro del contenedor ---
docker compose exec -w "/var/www/projects/$Nombre" web git init -q
# docker compose exec -w "/var/www/projects/$Nombre" web composer install -q

Write-Host "Proyecto '$Nombre' creado correctamente." -ForegroundColor Green
Write-Host "HTTP: http://$Nombre.localhost/"
Write-Host "HTTPS: https://$Nombre.localhost/"

```

Uso

```

cd C:\proyectos\docker-php
.\new-project.ps1 -Nombre proyecto2

```

En segundos: carpeta `proyecto2/public` con `index.php` y `.htaccess` activo, `composer.json` con PHPUnit, `src/`, `tests/`, repositorio Git inicializado, dependencias instaladas — y accesible **de inmediato** en `https://proyecto2.localhost/`, sin tocar Apache, sin recargar el contenedor y sin editar el fichero `hosts` de Windows. Esa es la gran ventaja de `VirtualDocumentRoot`: el propio nombre de la carpeta ya es la ruta.

Si PowerShell bloquea la ejecución de scripts la primera vez, ejecuta una sola vez (como administrador):
`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned.`

En las próximas prácticas modificaremos este archivo para que cree nuevas carpetas y archivos que son necesarios en nuestros proyectos más avanzados.

13. Trabajar con varios proyectos a la vez

- **PhpStorm y VS Code:** abre siempre la carpeta raíz `C:\proyectos\docker-php` (no cada proyecto suelto). Todas las subcarpetas de `projects\` comparten automáticamente el mismo intérprete PHP remoto (Docker) y el mismo Xdebug, configurados una sola vez en el paso 7.
- **VS Code – tarea rápida de PHPUnit** (`.vscode\tasks.json`), útil para no escribir el nombre del proyecto cada vez:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "PHPUnit (proyecto activo)",
      "type": "shell",
      "command": "docker compose exec -w /var/www/projects/${input:proyecto} web
phpunit tests",
      "problemMatcher": []
    }
  ],
  "inputs": [
    {
      "id": "proyecto",
      "type": "promptString",
      "description": "Nombre de la carpeta del proyecto"
    }
  ]
}
```

14. Chuleta de comandos docker

```
# Arrancar todo el entorno
docker compose up -d

# Parar el entorno (sin borrar nada)
docker compose stop

# Ver logs de Apache/PHP en directo
docker compose logs -f web

# Entrar a una terminal dentro del contenedor PHP
docker compose exec web bash

# Ejecutar Composer/PHPUnit en un proyecto concreto
docker compose exec -w /var/www/projects/NOMBRE web composer install
docker compose exec -w /var/www/projects/NOMBRE web phpunit tests

# Reconstruir la imagen tras cambiar el Dockerfile
docker compose up -d --build

# Borrar TODO (contenedores + volúmenes, incluida la base de datos)
docker compose down -v
```

15. Limitar la RAM que usa Docker (opcional, útil en portátiles con poca memoria)

Docker en Windows corre dentro de WSL2, que por defecto puede llegar a usar hasta la mitad de la RAM del equipo. Para acotarlo, crea `C:\Users\TU_USUARIO\.wslconfig`:

```
[wsl2]
memory=4GB
processors=2
```

Y ejecuta `wsl --shutdown` seguido de reabrir Docker Desktop. Para prácticas de clase (PHP + Apache + MySQL + phpMyAdmin), 3-4 GB es más que suficiente.

16. Checklist final de verificación

- `docker compose ps` muestra `php85_web`, `php85_db`, `php85_phpmyadmin` en `running`
- `docker compose exec web php -v` → PHP 8.5.x
- `docker compose exec web php -m` → aparecen `pdo_mysql`, `mysqli`, `xdebug`, `zip`
- `docker compose exec web apache2ctl -M` → aparece `vhost_alias_module` (este módulo es de Apache, no de PHP: no busques `vhost_alias` en `php -m`)
- `docker compose exec web composer -V` → última versión estable
- `docker compose exec web git --version` y `docker compose exec web unzip -v` funcionan
- `https://localhost/` muestra la página de aviso, sin error de certificado
- `http://localhost:8080` abre phpMyAdmin y conecta con `db`
- `.\new-project.ps1 -Nombre pruebas` crea el proyecto y `https://pruebas.localhost/` responde de inmediato, sin ningún paso adicional
- Desde `https://localhost/pruebas/composer.json` (sin subdominio) **no** se puede acceder — confirma que `public/` sigue aislado
- Un breakpoint en `public/index.php` se detiene tanto desde PhpStorm como desde VS Code
- El entorno se arranca **siempre** con `docker compose up -d` por terminal; la configuración "PHP Web Page" de PhpStorm solo abre el navegador y activa Xdebug, nunca sustituye a ese comando

17. Problemas habituales

Síntoma	Causa probable	Solución
<code>docker run hello-world</code> falla con <code>npip</code> / <code>dockerDesktopLinuxEngine</code>	El daemon aún no ha arrancado	<code>wsl --shutdown</code> , esperar 10 s, reabrir Docker Desktop
Error 404 al entrar a <code>https://proyecto.localhost/</code>	Falta <code>public/index.php</code> dentro de <code>projects\proyecto\public\</code>	Comprobar que existe; si se creó con <code>new-project.ps1</code> , revisar que el script terminó sin errores

Síntoma	Causa probable	Solución
El navegador muestra "Conexión no privada"	El certificado no incluye el comodín, o no se instaló la CA	Repetir 5.2 y 5.3 asegurando que se pasa <code>"*.localhost"</code> ; reiniciar el navegador
<code>https://localhost/proyecto/algo.php</code> sí responde (no debería)	El volumen de proyectos se montó en la misma ruta que el vhost catch-all	Revisar que <code>docker-compose.yml</code> monta <code>./projects</code> en <code>/var/www/projects</code> , no en <code>/var/www/html</code> ni <code>/var/www/default</code>
<code>.htaccess</code> no tiene efecto	Falta <code>AllowOverride All</code> en <code><Directory /var/www/projects></code>	Revisar 4.1/4.2
Xdebug no se conecta desde Windows	Firewall bloqueando el puerto 9003, o el editor no está "escuchando"	Permitir el puerto 9003 en el Firewall de Windows; revisar paso 7
Puerto 80/443 ocupado al hacer <code>docker compose up</code>	Otro servicio de Windows (IIS, Skype, otro Apache) usa ese puerto	Pararlo o cambiar el mapeo en <code>docker-compose.yml</code> (ej. <code>"8443:443"</code>)
phpMyAdmin no conecta	El contenedor <code>db</code> aún no ha terminado de arrancar	Esperar unos segundos, revisar <code>docker compose logs db</code>
Error de ejecución de scripts en PowerShell	Política de ejecución restringida por defecto en Windows	<code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> (una sola vez, como administrador)
No aparece <code>Settings → PHP → Servers</code> en el árbol de PhpStorm	En PhpStorm 2025.2+ esa página ya no vive en el árbol de Settings	Crear el servidor desde Run → Edit Configurations → PHP Web Page → botón ... junto a Server (ver paso 7.1)
Al pulsar ► en una configuración "PHP Web Page" de PhpStorm sale <code>Port 9003 is busy</code>	Ya había una escucha de Xdebug activa de una sesión anterior que no se cerró bien	Pulsa el icono del teléfono/auricular ("Start/Stop Listening for PHP Debug Connections") para apagar y volver a

Síntoma	Causa probable	Solución
		activar la escucha; no hace falta tocar Docker
<code>ERR_CONNECTION_REFUSED</code> al pulsar ► en una configuración de PhpStorm	Esa configuración solo abre el navegador; no arranca el contenedor	Comprobar primero <code>docker compose ps</code> por terminal y, si no está <code>running</code> , ejecutar <code>docker compose up -d</code> antes de nada

Con esta versión, cualquier compañero de clase que clone `C:\proyectos\docker-php` (sin la carpeta `projects` con código propio) y ejecute `docker compose up -d --build` tiene el mismo PHP 8.5, MySQL 8.4, Apache, Xdebug, Composer y HTTPS wildcard. Crear un proyecto nuevo con su propia URL real es un único comando (`.\new-project.ps1 -Nombre nombre`), y la carpeta `src/` de cada proyecto queda protegida del acceso web tanto por subdominio como por el dominio plano `localhost`.